

# ■ Day 3 — Data Modeling with EF Core

Recap card for Task Manager with Login (ASP.NET Core MVC). Companion to the 12-Day Build Recipe — read this after finishing Day 2, before opening Day 4.

---

## Goal for the day

End of day: **TaskItem.cs** and **ApplicationUser.cs** written by you, **ApplicationDbContext** wired to include them, first real migration created and applied, and you can see the **Tasks** table with the exact columns you designed.

■■ This is the recipe's own "highest-leverage day." Every later query and security check depends on this model being right — take the extra time here.

### Block 1 (0:00–0:15) — Reconnect

- Reread yesterday's **LEARNING.md**. Recite from memory: your 9 **TaskItem** fields, your DB choice, and why.
- Redraw diagram 1.4 (the ER diagram — **AspNetUsers** → owns → **TaskItem**) on paper without looking.
- If you can't reproduce it, that's today's warning sign, not a failure — fix it before moving on.

### Block 2 (0:15–1:30) — Type the model yourself

Work from the spec table (Section 3.1 of the Implementation Core Spec), but don't copy it verbatim — use it as a checklist while you decide the actual **C#** yourself.

- Create **Models/TaskItem.cs**. For each field, decide the type and whether it needs a data annotation (**[Required]**, **[StringLength]**, **[DataType]**, **[Display]**). The spec tells you which fields need annotations — you write the actual attribute syntax yourself, not by asking for the class.
- Create **Models/ApplicationUser.cs** extending **IdentityUser**, adding just **DisplayName** and **CreatedAt**, plus the inverse **ICollection<TaskItem> Tasks** navigation property.
- Stop and run **dotnet build**. Fix whatever breaks yourself first — this is the struggle that builds understanding.

■ Optional extra rep: try writing **IsOverdue** as a computed **[NotMapped]** property. Good first taste of "properties that aren't columns."

### Block 3 (1:30–2:00) — Wire the DbContext

- Open **Data/ApplicationDbContext.cs**. Confirm it currently inherits **IdentityDbContext<ApplicationUser>** (Identity scaffolding should have set this up — check).
- Add **DbSet<TaskItem> Tasks { get; set; }**.
- In **OnModelCreating**, configure the **TaskItem** entity: table name, required fields, max lengths, the foreign key relationship to **ApplicationUser**, and the indexes on **UserId**. Use the spec's Section 4 as a reference to check against, not something to retype from — try writing it from what you understand of Fluent API first, then compare.

■ Do not forget `base.OnModelCreating(builder)` — this is the #1 failure mode in the spec's own troubleshooting table (Identity tables go missing without it). Write down in `LEARNING.md` why this line matters.

## Block 4 (2:00–2:30) — Migrate and verify

```
dotnet ef migrations add InitialCreate
dotnet ef database update
```

- Open the DB in a SQLite browser (or run `PRAGMA table_info('Tasks');` if comfortable in CLI).
- Confirm every column you designed is there, with the type and constraints you expect.
- Run `PRAGMA foreign_key_list('Tasks');` — confirm it points to `AspNetUsers`.

If you get a `SqliteException` or something won't build — this is exactly where the "AI as rubber duck" rule applies.

## Block 5 (2:30–2:50) — AI as reviewer, not typist

Good questions to bring now:

- "Here's my `OnModelCreating` for `TaskItem` — does this actually enforce what I think it does?"
- "I got [error]. I think the cause is [your hypothesis]. Am I right?"
- "What actually happens if I change a field type after this migration is applied — walk me through it, don't fix it for me."

■ Bad question to avoid: "Write my `ApplicationDbContext.cs`."

## Block 6 (2:50–3:00) — Blank-Page Test + commit

- Close everything. From memory, list all 9 `TaskItem` columns and their types, and explain what the FK/cascade behavior does.
- Finish `LEARNING.md`.

```
git add .
git commit -m "add TaskItem and ApplicationUser models, first migration"
```

## Definition of Done — Day 3

- `TaskItem.cs` and `ApplicationUser.cs` written by you, with annotations
- `ApplicationDbContext` has `DbSet<TaskItem>` and full `OnModelCreating` config
- `InitialCreate` migration created and applied
- `Tasks` table exists in the DB with exact columns, verified by SQL browser/CLI
- You can explain the FK/cascade relationship and why `base.OnModelCreating` matters
- `LEARNING.md` updated, one git commit

## Don't do yet

No controller, no views, no CRUD logic — that's Day 4–5, and it needs today's table to exist first. Don't touch `TaskController.cs` at all today.

■ Core discipline reminder: diagrams are your map, the Explain-First Loop is your method, struggle-before-AI is your gate. Type everything yourself; use AI to explain, review, and challenge — never to produce.

Next: Day 4–5 — CRUD controllers & views